BrandStore — Phase 2, Task 1: API Contract Review and Application Plan

Item	Value
Author	Ahmed
Date	2026-11-12 (revision 2, after review)
Branch	`feature/api-contract-and-app-plan` (naming convention in section 10)
This report	`README.md` (this file) and the same content as `REPORT.pdf`
Run the site	`npm install`, then `npm run start` (http://127.0.0.1:8080) or `npm run dev` (Vite)
API contract	`api/openapi.json` + mock server `mock-server/server.mjs` (`npm run mock`, port 4010, demo login `demo@brandstore.test` / `Passw0rd!`)
Phase 1 documentation	`docs/phase1/README.md`, `task_1_build_plan.md`, `LAUNCH_REPORT.md`

1. What Changed Since the Last Review

Review point	What changed	Evidence
Required npm scripts (dev, build, lint, format, test, e2e)	Added `build` and `e2e` (plus `start` and `preview`). All scripts are listed in section 4 with what was verified	`package.json`
External ES modules, small named functions, no inline scripts or global sprawl	All site code moved from 11 classic scripts on a `window.BrandStore` global into ES modules in `src/`. Each page loads one module entry; no globals; every named function is commented and at most 40 lines	`qa/js-rules.js`: 0 failures (43 files, 85 functions, longest 38 lines)
Report did not match the site	One report (this file). Section 3 lists exactly what the site does today and what is only planned	sections 3 and 7
Required sections and tables	Endpoint Table (section 5), QA Matrix (section 7), Deviations List (section 12)	this file
Interactive features not reliable	Added 9 automated end-to-end journeys covering every interactive feature; re-ran all 41 Phase 1 behaviour checks after the move to modules	section 7: 17 of 17 runs and 41 of 41 checks pass

2. Answer to the Product Question

"How should shoppers browse, filter, and manage a cart and account so that they can trust prices, stock, and order history?"

- **Prices come from the API only, in integer cents.** No floating-point money anywhere. The cart keeps a price snapshot for display, but checkout re-reads `GET /products/{id}`, so a stale price is never charged. Formatting happens in one function (`utils/money.formatMoney`).
- **Stock is shown from the API's `stock` field.** Quantities are clamped to `min(stock, 10)`, out-of-stock items cannot be added, and the `inStock` filter is sent to the API, not applied to a stale list.
- **Order history is read from `GET /account/orders` every time and never stored on the device.** "Cancel" appears only when the API says `cancellable: true`. If the order changed meanwhile, the API answers 400 `ORDER_NOT_CANCELLABLE` and the list is refreshed.
- **Browsing state lives in the URL** (category, search, price range, sort, page), so links can be shared and Back works. Invalid values are corrected before a request is sent; a page past the end shows an empty state, not an error.
- **The cart and saved items survive refresh, closing the tab and logging out** (localStorage); **the sign-in survives refresh but not closing the tab** (sessionStorage). Any 401 signs the shopper out cleanly; a 429 shows how many seconds to wait.

3. What Is on the Site Today vs Planned

Feature	On the site today?	Code	Tested by
Product cards and testimonials rendered from data (3 products, 2 quotes)	**Yes**	`src/components/catalog-render.js`, data in `src/state/content.js`	e2e "home", `qa/behaviour.js`
Category filter buttons and category-card links, with a spoken result count	**Yes**	`src/components/product-filter.js`	e2e "browse" x2

Feature	On the site today?	Code	Tested by
Launch-offers strip with previous/next buttons	**Yes**	`src/components/deals.js`	e2e "deals"
Newsletter form with validation and accessible messages	**Yes** (nothing is sent)	`src/components/newsletter.js`, `src/utils/validate.js`	e2e "newsletter", unit tests
Mobile menu, skip link, keyboard-scrollable regions	**Yes**	`src/components/nav.js`, `scroll-regions.js`	e2e "keyboard", "phone menu"
Comparison table with "highlight differences"	**Yes**	`src/components/comparison-table.js`	e2e "comparison"
404 page	**Yes**	`404.html`, `src/pages/not-found.js`	e2e "not found"
Catalogue from the API: search, price range, sort, pagination	**No: planned** (task 2)	stubs in `src/api/products.js`, `src/state/catalog.js`, `src/utils/query-string.js`	contract checks the API now
Cart and save-for-later	**No: planned**	stubs in `src/state/cart.js`, `saved.js`, `src/utils/cart-math.js`	test plan section 9
Sign-in, order history, cancel order, address book	**No: planned**	stubs in `src/api/`, `src/state/session.js`, `src/components/`	contract checks the API now

The planned modules exist as documented stubs (JSDoc signatures that throw "planned but not implemented"), so the boundaries can be reviewed and linted before feature code is written. No page imports them yet.

4. npm Scripts

Script	Command	Verified in the build environment
`dev`	`vite`	No: Vite could not be installed here (no registry access); `npm run start` serves the same files
`build`	`node scripts/build.mjs`	**Yes**: copies the site to `dist/`, checks every page reference and that all 15 modules imported by the pages exist, writes `dist/_headers`; fails if anything is missing (checked by breaking an import)
`start` / `preview`	`node scripts/serve.mjs` (project root / `dist/`)	**Yes**
`lint`	`eslint .` (ESLint 10.1.0)	**Yes: 0 problems on a fresh clone**
`format` / `format:check`	`prettier --write .` / `prettier --check .` (Prettier 3.8.1)	**Yes: no changes / all files formatted on a fresh clone**
`test`	`vitest run`	37 unit tests pass through a Vitest-compatible shim on Node's test runner (Vitest not installable here)
`e2e`	`node tests/e2e/run.mjs` (Playwright library)	**Yes: 17 of 17 journey runs pass** (Chromium)
`contract`	`node scripts/contract.mjs`	**Yes: 45 requests, 0 problems**
`mock`	`node mock-server/server.mjs`	**Yes**
`qa`	Phase 1 static, accessibility and behaviour checks	**Yes**
`branch:check`	`node scripts/check-branch.mjs`	**Yes**
`check`	branch, lint, format, test, build, e2e, contract	as above

5. Endpoint Table (spec vs observed)

Generated by `npm run contract` (copied from `docs/api/ENDPOINTS.md`). Spec = `api/openapi.json`; observed = what the mock server returned.

Method	Path	Auth	Summary	Status codes in spec	Status codes observed	Error codes observed	Success JSON (top level; data fields)
GET	`/products`	none	List products with filters, sorting and pagination	200, 400, 429	200, 400, 429	VALIDATION_ERROR, RATE_LIMITED	{data: [id, name, category, price, compareAtPrice, currency, rating, reviewCount, stock, image], meta: {page: integer, limit: integer, total: integer, totalPages: integer}}
GET	`/products/{id}`	none	Get one product	200, 400, 404, 429	200, 400, 404, 429	NOT_FOUND, VALIDATION_ERROR, RATE_LIMITED	{data: id, name, category, price, compareAtPrice, currency, rating, reviewCount, stock, image, description, specs}

Method	Path	Auth	Summary	Status codes in spec	Status codes observed	Error codes observed	Success JSON (top level; data fields)
POST	`/auth/login`	none	Exchange email and password for a bearer token	200, 400, 401, 429	200, 400, 401, 429	VALIDATION_ERROR, INVALID_JSON, INVALID_CREDENTIALS, RATE_LIMITED, TOO_MANY_LOGIN_ATTEMPTS	{data: token, tokenType, expiresIn, user}
GET	`/account/orders`	Bearer	List the signed-in user's orders, newest first	200, 400, 401, 429	200, 400, 401, 429	UNAUTHENTICATED, INVALID_TOKEN, VALIDATION_ERROR, RATE_LIMITED	{data: [id, createdAt, status, items, subtotal, shipping, total, currency, cancellable, cancelledAt, cancelReason], meta: {page: integer, limit: integer, total: integer, totalPages: integer}}
PATCH	`/account/orders/{id}/cancel`	Bearer	Cancel an order that is still processing	200, 400, 401, 404, 429	200, 400, 401, 404, 429	UNAUTHENTICATED, VALIDATION_ERROR, ORDER_NOT_CANCELLABLE, NOT_FOUND, RATE_LIMITED	{data: id, createdAt, status, items, subtotal, shipping, total, currency, cancelledAt, cancelReason, cancellable}
GET	`/account/addresses`	Bearer	List saved addresses	200, 401, 429	200, 401, 429	UNAUTHENTICATED, RATE_LIMITED	{data: [id, label, fullName, line1, line2, city, postalCode, country, phone, isDefault]}
PUT	`/account/addresses/{id}`	Bearer	Create or replace the address with this client-chosen id (idempotent)	200, 201, 400, 401, 429	200, 201, 400, 401, 429	UNAUTHENTICATED, VALIDATION_ERROR, RATE_LIMITED	{data: id, label, fullName, line1, line2, city, postalCode, country, phone, isDefault}
DELETE	`/account/addresses/{id}`	Bearer	Delete an address	200, 401, 404, 429	200, 401, 404, 429	UNAUTHENTICATED, NOT_FOUND, RATE_LIMITED	{data: id, deleted}

Error body for every 4xx: `{ "error": { "code", "message", "details"?: [{ "field", "message" }] } }`. Every response has `X-RateLimit-Limit` and `X-RateLimit-Remaining`; 429 adds `Retry-After`; 401 adds `WWW-Authenticate: Bearer`; 201 adds `Location`. Outside the 8 operations: unknown path returns 404, wrong method 405 with `Allow`, and `OPTIONS` returns 204 (CORS).

Status	How it was produced (full log of 45 requests: `docs/api/ENDPOINTS.md`)
200	Valid requests; `GET /products?page=99` returns 200 with empty `data`; PUT on an existing address id
201	`PUT /account/addresses/office` when `office` did not exist (with `Location`)
400	`limit=500`; `minPrice > maxPrice`; unknown `category`/`sort`; bad product id; login without fields; malformed JSON; `status=lost`; cancel reason over 200 characters; cancelling a delivered or already cancelled order (`ORDER_NOT_CANCELLABLE`); invalid address fields
401	`/account/*` without a token (`UNAUTHENTICATED`) or with an unknown token (`INVALID_TOKEN`); wrong password (`INVALID_CREDENTIALS`)
404	Unknown product; unknown order `o-9999`; deleting an address twice
429	The 61st request within 60 s from one client, on every operation (`RATE_LIMITED`); the 6th failed login for one email within 60 s (`TOO_MANY_LOGIN_ATTEMPTS`)

6. JSON Shapes Observed

```
GET /products                   { data: ProductSummary[], meta: { page, limit, total, totalPages } }
ProductSummary                  { id, name, category, price (cents), compareAtPrice|null, currency, rating, reviewCount, stock, image: { url, alt } }
GET /products/{id}              { data: ProductSummary + { description, specs: [{ label, value }] } }
POST /auth/login                { data: { token, tokenType: "Bearer", expiresIn: 3600, user: { id, name, email } } }
GET /account/orders             { data: Order[], meta }
Order                           { id, createdAt, status, items: [{ productId, name, quantity, unitPrice, lineTotal }], subtotal, shipping, total, currency, cancellable,
cancelledAt|null, cancelReason|null }
PATCH /account/orders/{id}/cancel  { data: Order }
GET /account/addresses          { data: Address[] }
Address                         { id, label, fullName, line1, line2|null, city, postalCode, country, phone|null, isDefault }
PUT /account/addresses/{id}     { data: Address }   (201 created, 200 replaced)
DELETE /account/addresses/{id}  { data: { id, deleted: true } }
```

7. QA Matrix

End-to-end journeys (browser x viewport), `npm run e2e`, 2026-11-12:

Journey	Chromium 1280x900	Chromium 375x740	Firefox	WebKit (Safari)
Home renders 3 products and 2 testimonials; no `window.BrandStore` global	Pass	Pass	Not run	Not run
Category filter shows matching products and announces the count	Pass	Pass	Not run	Not run
Category card link filters the list	Pass	Pass	Not run	Not run
Deals: next scrolls, previous disabled at start	Pass	Pass	Not run	Not run
Newsletter: invalid shows error; valid shows success, and the old error does not come back	Pass	Pass	Not run	Not run
First Tab is the skip link; Enter moves focus to main	Pass	Pass	Not run	Not run
Phone menu opens, Escape closes, focus returns to the button	n/a (phone only)	Pass	Not run	Not run
Comparison table: highlight differences on and off	Pass	Pass	Not run	Not run
Unknown address returns the 404 page with one h1	Pass	Pass	Not run	Not run

Firefox and WebKit are not installed in the build environment. To run them: `npx playwright install firefox webkit`, then `E2E_BROWSERS=chromium,firefox,webkit npm run e2e`.

All other checks, re-run after the move to ES modules:

Check	Command	Environment	Result
Lint	`npm run lint`	fresh clone	Pass: 0 problems
Formatting	`npm run format:check`	fresh clone	Pass
Module rules (ES modules, no inline scripts, no globals, functions ≤ 40 lines)	`node qa/js-rules.js`	static	Pass: 0 failures
Unit tests	`npm test`	Node test runner + shim	37 of 37 pass
Build	`npm run build`	Node 22	Pass: 15 modules resolved, 0 missing references
API contract	`npm run contract`	mock server	45 requests, 0 problems
Phase 1 behaviour (rendering, XSS safety, filter, menu, newsletter)	`node qa/behaviour.js`	Chromium	41 of 41 pass
Layout and overflow at 375, 768, 1024 and 1440 px	`node qa/layout.js`	Chromium	Pass
Keyboard order and focus	`node qa/keyboard.js`	Chromium	Pass
Device profiles (iPhone SE, iPhone 13, Pixel 7, iPad Mini, 2 desktops)	`node qa/matrix.js`	Chromium emulation	6 of 6 pass
Accessibility and SEO audit (own rules, not Lighthouse)	`node qa/preflight.js`	Chromium	163 of 163 pass
Text contrast	`node qa/contrast.js`	Chromium	Lowest 5.02:1 (needs 4.5:1)
Layout shift	`node qa/cls.js`	Chromium, CPU x6	CLS 0
Security headers and CSP (module scripts allowed by `script-src 'self'`)	`node qa/headers.js`	Chromium + Netlify headers	Pass: 0 failures

8. Module Layout (ES modules)

Folder	Owns	May import	Must not
`src/pages/`	One entry module per HTML page; picks the features and calls `start()`	`main.js`, `components/`	contain feature logic
`src/main.js`	`start(features)`: runs each feature, isolates failures; later wires the http client and stores	everything	contain business logic
`src/components/`	One piece of UI. Live features export `init()`; planned API components export `mount(root, deps)`	`api/`, `state/`, `utils/`	import `main.js`
`src/state/`	What the app remembers: today the static content; planned cart, saved items, session, catalog filters	`utils/`	call the API
`src/api/`	The REST API: one fetch wrapper (`http.js`), one file per resource, `ApiError`	`utils/`	touch the DOM or state

Folder	Owns	May import	Must not
`src/utils/`	Pure helpers: DOM, validation, money, cart math, query strings, storage	nothing from the app	call `fetch` or read state

Dependency direction: `pages -> main.js, components -> (api, state) -> utils`. ESLint enforces it (restricted imports per folder; `fetch` only in `api/http.js`; Web Storage only in `utils/storage.js`; checked with probe files: 4 of 4 violations reported). `qa/js-rules.js` enforces the page rules. Full map and flows: `docs/ARCHITECTURE.md`.

9. Test Plan (summary)

Highest-risk module	Unit	Integration (Vitest + MSW)	End-to-end
Cart math (`utils/cart-math.js`, `state/cart.js`)	Every function, table-driven, cents only, shipping threshold 9999/10000	Cart store + storage persistence, corrupt storage ignored	Add, change quantity, refresh, totals unchanged
Auth (`api/auth.js`, `api/http.js`, `state/session.js`)	Session expiry with a fake clock; sign-out clears the right keys	Bearer header; 401 signs out once; 429 lockout message	Sign in, refresh, sign out
Query-string building (`utils/query-string.js`, `state/catalog.js`)	Build/parse, encoding, defaults, round trip	Exact request URL seen by MSW; 400 details shown	Filter, paginate, Back, shared link

All modules, cases and exit criteria: `docs/TEST_PLAN.md`.

10. Feature Branch and Naming Convention

`<type>/<short-description>`: type is one of `feature`, `fix`, `docs`, `chore`, `test`, `refactor`; lowercase kebab-case; at most 50 characters; branched from `main`; merged by pull request with CI green. This work: **`feature/api-contract-and-app-plan`** (`npm run branch:check`). Details: `CONTRIBUTING.md`.

11. Storage Strategy

Data	Where	Key	Refresh	Tab close	Logout	Why
Cart lines (`productId`, `quantity`, price snapshot)	localStorage	`brandstore:cart:v1`	kept	kept	kept	A shopper expects the cart tomorrow and after signing out/in. It holds no personal data. Prices are re-checked against `GET /products/{id}` before checkout, so a stale snapshot cannot be charged.
Save-for-later items	localStorage	`brandstore:saved:v1`	kept	kept	kept	A wishlist is only useful if it lasts; same content and risk level as the cart.
Session: token, `expiresAt`, user `{id, name, email}`	sessionStorage	`brandstore:session:v1`	kept	**cleared**	**cleared**	Refresh must not sign the user out, but a bearer token should not outlive the tab (shared computers). Cleared on logout and on any 401.
Transient flags: `returnTo` after login, "session expired" notice, dismissed rate-limit banner, last cancelled order id for a one-time message	sessionStorage	`brandstore:flags:v1`	kept	cleared	**cleared**	Only meaningful during this visit; must not leak into a later visit or another user's session.
Catalog filters, sort, page	URL query string	n/a	kept	n/a (in history and bookmarks)	unaffected	Shareable and back/forward work; storing them would make the URL lie.
Products, orders, addresses from the API	memory only	n/a	refetched	gone	cleared	The server is the source of truth. Orders and addresses are personal data and are never written to the device.

Logout removes the session and flags and drops account data from memory; it keeps the cart and saved items. Reasons and trade-offs: `docs/STORAGE.md`.

12. Deviations List

#	Deviation	Reason	Follow-up
1	`api/openapi.json` and `mock-server/` were written for this task	No OpenAPI file or mock server was provided	`SPEC=<file> API_URL=<server> npm run contract` re-checks the official ones; then regenerate section 5
2	Endpoints exercised by a script (Node `fetch`), not by hand in a REST client	Repeatable evidence for every status code	`docs/api/brandstore.postman_collection.json` runs the same 36 requests in Postman or Insomnia
3	The site must be served over http; opening `index.html` from disk no longer runs the scripts	Browsers do not load ES modules from `file://`	`npm run start`, `npm run dev`, or any host
4	`build` copies and verifies files instead of bundling	Vite is allowed as a dev server only; browsers load the modules natively	Revisit if the module count makes loading slow
5	`e2e` uses the Playwright library with a small runner, not `@playwright/test`	It could be run and verified offline here	Same journeys can move to `@playwright/test` unchanged in logic
6	No `package-lock.json`; Vite, Vitest, MSW not installed; unit tests run through a shim	No package-registry access in the build environment	`npm install`, commit the lock file, run `npm test`
7	`PUT` and `DELETE` addresses take an id: `/account/addresses/{id}`	A collection-level `PUT`/`DELETE` would replace or wipe every address	Confirm with the official spec
8	Non-cancellable order returns 400 (409 would be more usual); `DELETE` returns 200 with a body, not 204	Stay within the status codes listed for this task	Adjust to the official spec
9	405 (wrong method) and 204 (`OPTIONS`) exist beyond the listed codes	Correct HTTP behaviour	none
10	Phase 1 HTML and CSS are linted where applicable but excluded from Prettier	Reformatting would bury their history in whitespace changes	Remove from `.prettierignore` when each page is reworked
11	The Content-Security-Policy (`default-src 'self'`) will block an API on another origin	Found while planning `api/http.js`	Add the API origin to `connect-src` when the host is known
12	The 404 page loads its CSS and modules by relative path, so a deep wrong address such as `/a/b` shows it unstyled	Relative paths keep the site working from a sub-folder (GitHub Pages)	Root-relative paths once the host is fixed

13. How to Verify

```
git checkout feature/api-contract-and-app-plan
npm install
npx playwright install chromium
npm run lint && npm run format:check && npm test && npm run build && npm run e2e && npm run contract
npm run start     # open http://127.0.0.1:8080
```